12
17/07/26

End - End Communication at Transport Layer

a) Implement echo client server using TCP/~~IP~~ UDP Sockets

AIM :-

To implement an Echo client - server application using TCP Sockets and verify communication between the client and server.

Steps :

1.) Create a TCP Echo server using Server Socket and Socket
2.) Assign a port number to the server
3.) Compile and execute the TCP server program
4.) Create a TCP echo client using Socket
5.) Connect the client to the server using the server address and port number.
6.) Enter a message through the client
7.) The server receives the message and sends it back to the client.
8.) Display the echoed message at the client
9.) Enter bye to terminal the communication.

RESULT :-

Thus, the TCP echo - client - server application was successfully implemented and the client successfully received the echoed message from the server.

a.) Program :-

TCP Echo Server.Java :-

```
import java.io.*;
import java.net.*;
public class TCP Echo Server {
    public ~~class~~ static void main (String[] args)
throws exception {
    Server Socket ss = new ServerSocket (5000));
    Socket s = ss.accept ();
    Buffered Reader in = new Buffered Reader ;
    String msg :
    while ((msg = in.readline ()) != null) {
        Out.println ("Echo" + msg);
        if (msg.equals Ignore Case ("bye")) breaks ;
    }
        s.close ();
        ss.close ();
    }
```

TCD Echo client.java :-

```
import java.io.*;
import java.net.*;
Public class TCP Echo client {
    Public static void main (String [] args) {
    Socket s = new Socket ("local host", 5000);
    Buffered Reader in = new Buffered Reader (
    print Writer Out = new Printer Writer ( s.get output Stream(),
                                            true);
    Buffered Reader Kb = new Buffered Reader ;
    String msg :
    while (!(msg = Kb.readline ()).equals
    System.out.println (in.readline ());
}
    s.close ();}
}
```

b.) Echo Client - Server using UDP

AIM:-

To implement an Echo Client - Server application using UDP sockets and verify communication between the client and server.

Procedure :-

1.) Create a UDP Echo server using Datagram socket and Datagram packet.
2.) Assign a port number to the UDP Server
3.) Compile and execute the UDP server program
4.) Create a UDP echo client using Datagram socket and datagram packet.
5.) Specify the server address and port number.
6.) Enter a message through the client
7.) The client sends the message as a UDP datagram to the server.
8.) The server receives the datagram and sends the echoed message back.
9.) Display the received echoed message at the client.
10.) Enter bytes to terminate the communication.

RESULT:-

Thus, the UDP Echo Client - Server application was successfully implemented, and the client successfully received the echoed message from the server.

4) Program :-

```
import java.net.*;
Public class UDP Echo Server {
    public static void main (String() args)
throws exception {
 Datagram socket s = new Datagram Socket (6000);
    while (true) {
 Datagram Packet P = new Datagram Packet (b, b.length),
  S.receive (P);
  S.send (new Datagram Packet (a, a.length)
      P.get Address (), P.get Port ());
  if (msg.equalsIgnore Case ("bye")) break; }
      S.close(); {
 }
```

UDP Echo Client.java

```
import java.net.*;
import java.io.*;
Public class UDP Echo client {
    Public static void main (String() args)
throws Exception {
    Datagram Socket s = new Datagram Socket ();
    Inet Address a = Inet Address.get by Name ("Local host");
    string msg;
 while (! msg = Kb.readline ()).equals
 Ignore Case ("Bye")) {
     S.Send (new Datagram Packet (b, b.length));
     S.receive (P);
     System.out.println (new string (P.get Data (),0,
         P.get Length ());
  }
     S.close ()
 }
}
```

Chat client - Server using TCP / UDP Sockets

AIM :-

To implement a chat client-server application using sockets and enable communication between multiple clients through a server.

Procedure :-

1) Create a chat server program using socket programming
2) Start and run the chat server.
3) Create the chat client program and connect it to the server.
4) Run Two or more client programs in separate terminals.
5) Enter a name of each client
6) Send messages from one client to the server
7) The server broadcasts the message to the other connected clients.
8) Verify that the clients can exchange message through the server.
9) Continue that communication between the clients.
10) Enter /quit to terminates a client connection.

RESULT :-

Thus, the chat client - Server application was successfully implemented, and message was successfully exchanged between multiple clients through the server.

c) Program :-

```
import java.io.*;
import java.util.*;
public class chat server {
    Static set< printWriter > clients = collections.synchronized
 set (new Hashset <>());
public static void main (String [] args)
 throws Exception {
     Server Socket ss = new Server Socket (5000);
     System.out.println ("Chat Server statement")
while (True) {
     Socket s = ss.accept ();
}
}
  Clients.remove (out);
  S.close ();
} catch (exception) {}
}

Chat Client.Java;
  import java.io.*;
  import java.net.*;
  public class chat client {
    public static void main (String [] args) throws
 Exception {
    Socket S = new Socket ("127.0.0.1", 5000);
new Thread () -> {
   Try {
        String m;
        while ((m = in.readline ()) != null)
        System.out.println (m); {
        Catch (Exception) {}
    System.out.println ( msg).
}
   S.close ();
}
```